SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Murukuti Poli reddy	**Reg. No.:**	192325056
Section / Batch:	AIML-2023	**Date:**	12/10/2026

Code of Conduct

I M.POLIREDDY/192325056(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.POLIREDDY

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Section-1:

Python-Based Resume Information Extraction System Using Regular Expressions

1. Aim

To design and implement a Python program using Regular Expressions (`re` module) to automatically extract candidate information from resumes in text format, generate structured candidate profiles, and shortlist candidates who meet the minimum eligibility criteria:

- Minimum 2 years of experience
- Must possess the Python skill

2. Algorithm

1. Read the resume text.
2. Extract the candidate's name using a regular expression.
3. Identify the email address.
4. Extract the mobile number.
5. Detect technical skills such as Python, Java, SQL, Machine Learning, and NLP.
6. Extract years of experience.
7. Generate a structured summary of each candidate.
8. Shortlist candidates who have at least 2 years of experience and Python as a skill.
9. Display the extracted profiles and eligible candidates.

3. Python Program

```
10.
11. !pip -q install gradio pymupdf python-docx matplotlib
12.
13. import re
14. import os
15. import gradio as gr
16. import matplotlib.pyplot as plt
17. import fitz
18. from docx import Document
19.
20. SKILLS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
21.
22. # Read uploaded PDF, DOCX or TXT resume
23. def read_resume(file_path):
24.     if not file_path:
25.         raise ValueError("Please upload a resume file.")
26.
27.     ext = os.path.splitext(file_path)[1].lower()
28.
29.     if ext == ".pdf":
30.         with fitz.open(file_path) as pdf:
31.             text = "\n".join(page.get_text() for page in pdf)
32.
33.     elif ext == ".docx":
34.         doc = Document(file_path)
35.         text = "\n".join(p.text for p in doc.paragraphs)
```

```
        for table in doc.tables:
            for row in table.rows:
                text += "\n" + " ".join(cell.text for cell in row.cells)

    elif ext == ".txt":
        with open(file_path, "r", encoding="utf-8",
                  errors="ignore") as f:
            text = f.read()

    else:
        raise ValueError("Please upload a PDF, DOCX or TXT file.")

    if not text.strip():
        raise ValueError(
            "No readable text found. If this is a scanned PDF, "
            "OCR may be required."
        )

    return text


def screen_resume(file_path):
    try:
        text = read_resume(file_path)

        # Extract candidate name
        name_match = re.search(
            r"(?im)^\s*(?:candidate\s+)?name\s*[:\-]\s*(.+?)\s*$",
            text
        )

        if name_match:
            name = name_match.group(1).strip()
        else:
            # Fallback: first non-empty line
            lines = [line.strip() for line in text.splitlines()
                     if line.strip()]
            name = lines[0][:80] if lines else "Not Found"

        # Extract email
        email_match = re.search(
            r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
            text
        )
        email = email_match.group(0) if email_match else "Not Found"

        # Extract Indian mobile number
        mobile_match = re.search(
            r"(?<!\d)(?:\+91[\s-]?)?[6-9]\d{9}(?!\d)",
            text
        )
        mobile = mobile_match.group(0) if mobile_match else "Not Found"

        # Detect technical skills
        skills = []
        for skill in SKILLS:
            pattern = r"(?<!\w)" + re.escape(skill) + r"(?!\w)"
            if re.search(pattern, text, re.IGNORECASE):
                skills.append(skill)

        # Extract experience
        exp_patterns = [
            r"(\d+(?:\.\d+)?)\s*\+?\s*years?\s+of\s+experience",
```

```
            r"experience\s*[:\-]\s*(\d+(?:\.\d+)?)\s*\+?\s*years?",
            r"(\d+(?:\.\d+)?)\s*\+?\s*years?\s+(?:of\s+)?(?:professional\s+)?experience"
               ]

               experience = None
               for pattern in exp_patterns:
                   match = re.search(pattern, text, re.IGNORECASE)
                   if match:
                       experience = float(match.group(1))
                       break

               # Eligibility decision
               has_python = "Python" in skills
               eligible = (
                   experience is not None
                   and experience >= 2
                   and has_python
               )

               if eligible:
                   status = "ELIGIBLE"
                   reason = "Meets both eligibility requirements."
               else:
                   status = "NOT ELIGIBLE"
                   reasons = []
                   if experience is None:
                       reasons.append("Experience could not be identified.")
                   elif experience < 2:
                       reasons.append("Less than 2 years of experience.")
                   if not has_python:
                       reasons.append("Python skill was not detected.")
                   reason = " ".join(reasons)

               experience_display = (
                   f"{experience:g} years"
                   if experience is not None else "Not identified"
               )

               # Candidate summary
               summary = f"""
       # Resume Screening Result

       ## Candidate Profile

       | Field | Extracted Information |
       |---|---|
       | Name | {name} |
       | Email | {email} |
       | Mobile | {mobile} |
       | Technical Skills | {", ".join(skills) if skills else "None detected"} |
       | Experience | {experience_display} |
       | Python Skill | {"Yes" if has_python else "No"} |
       | Eligibility | **{status}** |

       **Reason:** {reason}

       ### Eligibility Criteria
       - Minimum 2 years of experience
       - Python skill must be present
       """

               # Graphical output
               fig, ax = plt.subplots(figsize=(7, 3.5))
```

```
161.
162.             if experience is not None:
163.                 ax.barh(
164.                     ["Candidate"],
165.                     [experience],
166.                     color="green" if eligible else "tomato"
167.                 )
168.                 ax.axvline(
169.                     2, color="blue", linestyle="--",
170.                     label="Minimum 2 years"
171.                 )
172.                 ax.set_xlim(0, max(3, experience + 1))
173.                 ax.set_xlabel("Years of Experience")
174.                 ax.legend()
175.             else:
176.                 ax.text(
177.                     0.5, 0.5, "Experience not found in resume",
178.                     ha="center", va="center",
179.                     transform=ax.transAxes
180.                 )
181.                 ax.set_xticks([])
182.                 ax.set_yticks([])
183.
184.             ax.set_title("Candidate Eligibility Analysis")
185.             plt.tight_layout()
186.
187.             return summary, fig, text
188.
189.         except Exception as e:
190.             return f"**Error:** {str(e)}", None, ""
191.

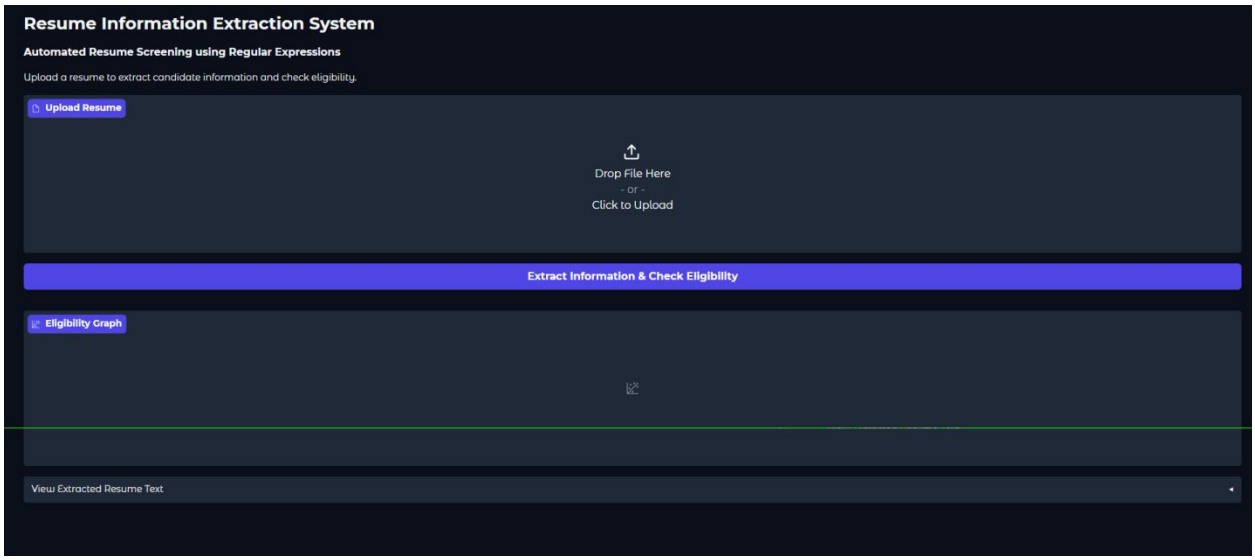
192.     # Website interface
193.     with gr.Blocks(theme=gr.themes.Soft()) as demo:
194.
195.         gr.Markdown("""
196.         # Resume Information Extraction System
197.         ### Automated Resume Screening using Regular Expressions
198.
199.         Upload a resume to extract candidate information and check eligibility.
200.         """)
201.
202.         file_input = gr.File(
203.             label="Upload Resume",
204.             file_types=[".pdf", ".docx", ".txt"],
205.             type="filepath"
206.         )
207.
208.         check_button = gr.Button(
209.             "Extract Information & Check Eligibility",
210.             variant="primary"
211.         )
212.
213.         result_output = gr.Markdown(label="Candidate Profile")
214.         chart_output = gr.Plot(label="Eligibility Graph")
215.
216.         with gr.Accordion("View Extracted Resume Text", open=False):
217.             text_output = gr.Textbox(
218.                 label="Resume Text",
219.                 lines=12
220.             )
221.
222.         check_button.click(
```

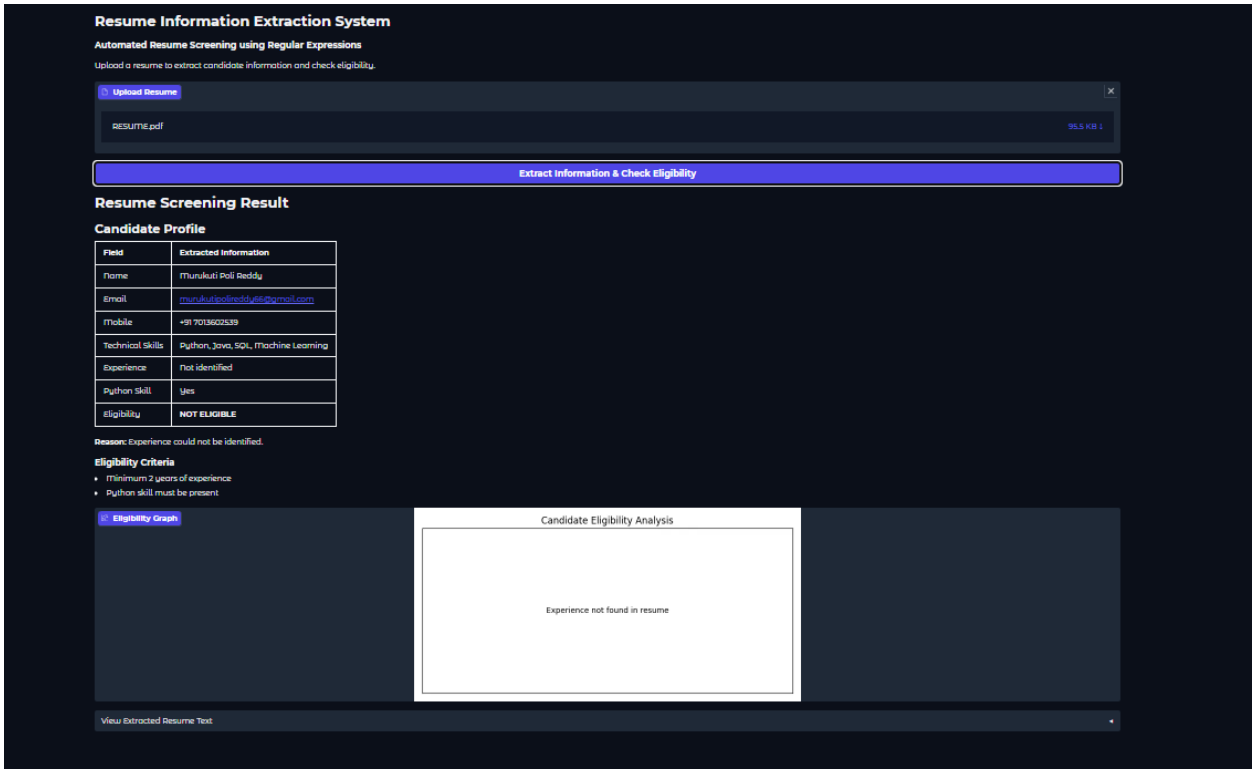
```
            fn=screen_resume,
            inputs=file_input,
            outputs=[result_output, chart_output, text_output]
        )

    demo.launch(share=True)

```

4. Sample Output





Section-2:

Python-Based Product Search System Using Regular Expressions

1. Aim

To design and implement a Python-based product search system using the `re` module that supports flexible keyword searches and generates a report showing the matching products and the total number of matches for each search.

2. Algorithm

1. Create a list of product names.
2. Accept a search keyword and search type.
3. Use Regular Expressions to perform the selected search:
    - Exact keyword search
    - Prefix-based search
    - Suffix-based search
    - Partial keyword search
    - Case-insensitive search
4. Display all matching product names.
5. Count the matching products.
6. Generate a search report containing the search type, keyword, matching products, and total count.

3. Python Program

```
!pip -q install gradio

import re
import gradio as gr

# Sample product database
products = [
    "Apple iPhone 15",
    "Samsung Galaxy S24",
    "Apple MacBook Air",
    "Dell Laptop",
    "HP Laptop",
    "Wireless Mouse",
    "Wireless Keyboard",
    "Bluetooth Headphones",
    "Sony Headphones",
    "Samsung Smart TV",
    "Apple Watch",
    "Gaming Laptop",
    "USB Cable",
    "Power Bank",
    "Smart Watch"
]

def search_products(keyword, search_type):
    if not keyword or not keyword.strip():
        return "Please enter a search keyword.", ""

    keyword = keyword.strip()
```

```
    # Build regular expression according to search type
    if search_type == "Exact Keyword":
        pattern = r"\b" + re.escape(keyword) + r"\b"

    elif search_type == "Prefix Search":
        pattern = r"\b" + re.escape(keyword)

    elif search_type == "Suffix Search":
        pattern = re.escape(keyword) + r"\b"

    elif search_type == "Partial Keyword":
        pattern = re.escape(keyword)

    else:  # Case-insensitive search
        pattern = re.escape(keyword)

    # Match products
    matches = [
        product for product in products
        if re.search(pattern, product, re.IGNORECASE)
    ]

    # Generate report
    if matches:
        result = "## Matching Products\n\n"
        for i, product in enumerate(matches, 1):
            result += f"{i}. {product}\n\n"
    else:
        result = "## No matching products found."

    report = f"""
    ### Search Report
    - **Keyword:** {keyword}
    - **Search Type:** {search_type}
    - **Total Matching Products:** {len(matches)}
    - **Total Products in Database:** {len(products)}
    """

    return result, report


# Website interface
with gr.Blocks(theme=gr.themes.Soft()) as demo:

    gr.Markdown("""
    # E-Commerce Product Search Engine
    ### Flexible Keyword Matching using Regular Expressions

    Search products by exact words, prefixes, suffixes, or partial keywords.
    """)

    keyword = gr.Textbox(
        label="Enter Product Keyword",
        placeholder="Example: Apple, Lap, phone..."
    )

    search_type = gr.Radio(
        choices=[
            "Exact Keyword",
            "Prefix Search",
            "Suffix Search",
            "Partial Keyword",
```

```
            "Case-Insensitive Search"
        ],
        value="Partial Keyword",
        label="Select Search Type"
    )

    search_button = gr.Button("Search Products", variant="primary")

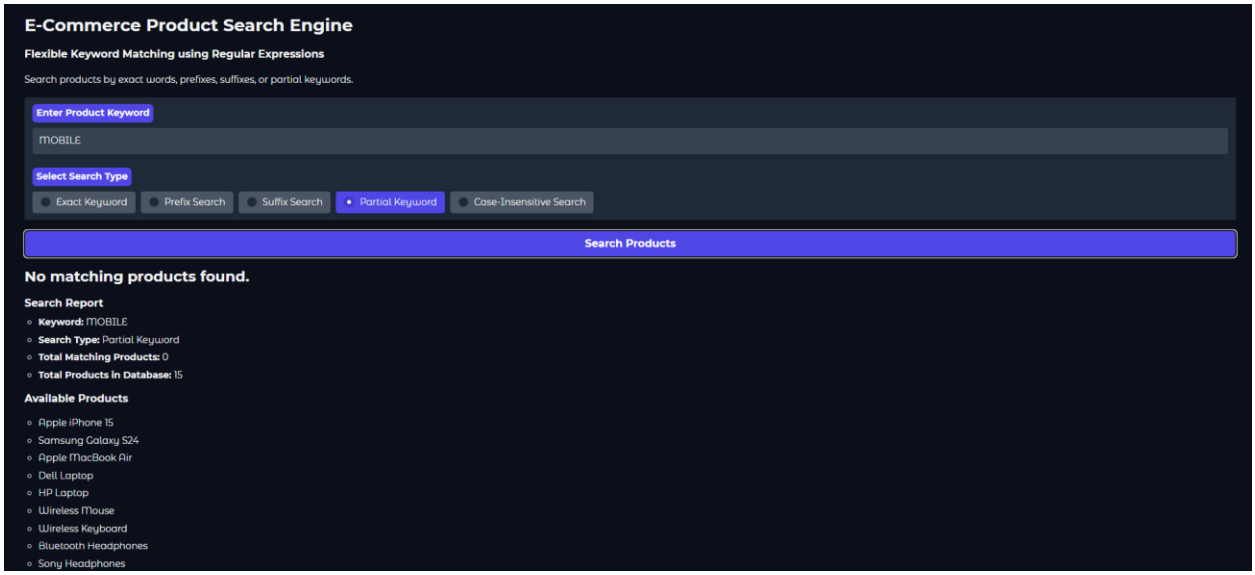
    results = gr.Markdown(label="Results")
    report = gr.Markdown(label="Search Report")

    search_button.click(
        fn=search_products,
        inputs=[keyword, search_type],
        outputs=[results, report]
    )

    gr.Markdown("### Available Products")
    gr.Markdown("\n".join(f"- {p}" for p in products))

demo.launch(share=True)
```

5. Sample Output



Python-Based Student Registration Validation System Using Regular Expressions

1. Aim

To design and implement a Python program using the `re` module to validate student registration details before course enrollment and generate a final registration status report.

The system validates:

- Student register number
- Institutional email address
- Course code
- Semester information
- Student mobile number

2. Algorithm

1. Start the program.
2. Accept student registration details.
3. Validate the register number using a regular expression.
4. Validate the institutional email address.
5. Validate the course code.
6. Validate the semester information.
7. Validate the mobile number.
8. Display a success or failure message for each field.
9. Generate the final registration status report.
10. Declare registration successful only if all fields are valid.

3. Python Program

```
!pip -q install -U gradio

import re
import gradio as gr

def validate_registration(reg_no, email, course, semester, mobile):
    fields = {
        "Register Number": (
            reg_no.strip().upper(),
            r"\d{2}[A-Z]{2,3}\d{3,4}"
        ),
        "Institutional Email": (
            email.strip(),
            r"[A-Za-z0-9._%+-]+@([A-Za-z0-9-]+\.)*university\.edu"
        ),
        "Course Code": (
            course.strip().upper(),
            r"[A-Z]{2,4}\d{3,4}"
        ),
```

```
        "Semester": (
            str(semester).strip(),
            r"[1-8]"
        ),
        "Mobile Number": (
            mobile.strip(),
            r"[6-9]\d{9}"
        )
    }

    results = []
    all_valid = True

    for field, (value, pattern) in fields.items():
        valid = bool(re.fullmatch(pattern, value))
        status = "□ VALID" if valid else "□ INVALID"
        results.append(
            f"| {field} | {value or 'Empty'} | {status} |"
        )
        if not valid:
            all_valid = False

    report = (
        "## Validation Report\n\n"
        "| Field | Entered Value | Status |\n"
        "|---|---|---|\n"
        + "\n".join(results)
    )

    if all_valid:
        report += (
            "\n\n## □ Registration Successful\n\n"
            "All fields passed validation."
        )
    else:
        report += (
            "\n\n## □ Registration Failed\n\n"
            "Correct the invalid fields and try again."
        )

    return report


custom_css = """
.gradio-container {
    max-width: 950px !important;
    margin: auto !important;
    background: #f0fdfa !important;
    font-family: Arial, sans-serif !important;
}

#main-title {
    background: linear-gradient(135deg, #134e4a, #0f766e);
    color: white;
    padding: 24px;
    border-radius: 15px;
    text-align: center;
    margin-bottom: 20px;
}

#main-title h1 {
    color: white !important;
}
```

```
#main-title p {
    color: #ccfbf1 !important;
}

.form-card {
    background: white !important;
    border: 1px solid #99f6e4 !important;
    border-radius: 14px !important;
    padding: 22px !important;
    box-shadow: 0 4px 15px rgba(15, 118, 110, .08);
}

label span {
    color: #134e4a !important;
    font-weight: 600 !important;
}

input {
    border-radius: 8px !important;
}

.primary-btn {
    background: #0f766e !important;
    color: white !important;
    border: none !important;
    border-radius: 9px !important;
    min-height: 46px !important;
    font-weight: bold !important;
}

.primary-btn:hover {
    background: #115e59 !important;
}

#result-box {
    background: white !important;
    border: 1px solid #99f6e4 !important;
    border-radius: 14px !important;
    padding: 20px !important;
    margin-top: 18px !important;
}
"""

with gr.Blocks(
    theme=gr.themes.Soft(primary_hue="teal"),
    css=custom_css
) as demo:

    gr.Markdown(
        "# University Registration Portal\n"
        "Validate student details before course enrollment.",
        elem_id="main-title"
    )

    with gr.Column(elem_classes=["form-card"]):
        gr.Markdown("### Student Information")

        reg_no = gr.Textbox(
            label="Student Register Number",
            placeholder="Example: 23AIML1234"
        )
```

```
        email = gr.Textbox(
            label="Institutional Email",
            placeholder="student@university.edu"
        )

        course = gr.Textbox(
            label="Course Code",
            placeholder="Example: AIML101"
        )

        semester = gr.Dropdown(
            choices=["1", "2", "3", "4", "5", "6", "7", "8"],
            label="Select Semester",
            value="1"
        )

        mobile = gr.Textbox(
            label="Mobile Number",
            placeholder="10-digit mobile number"
        )

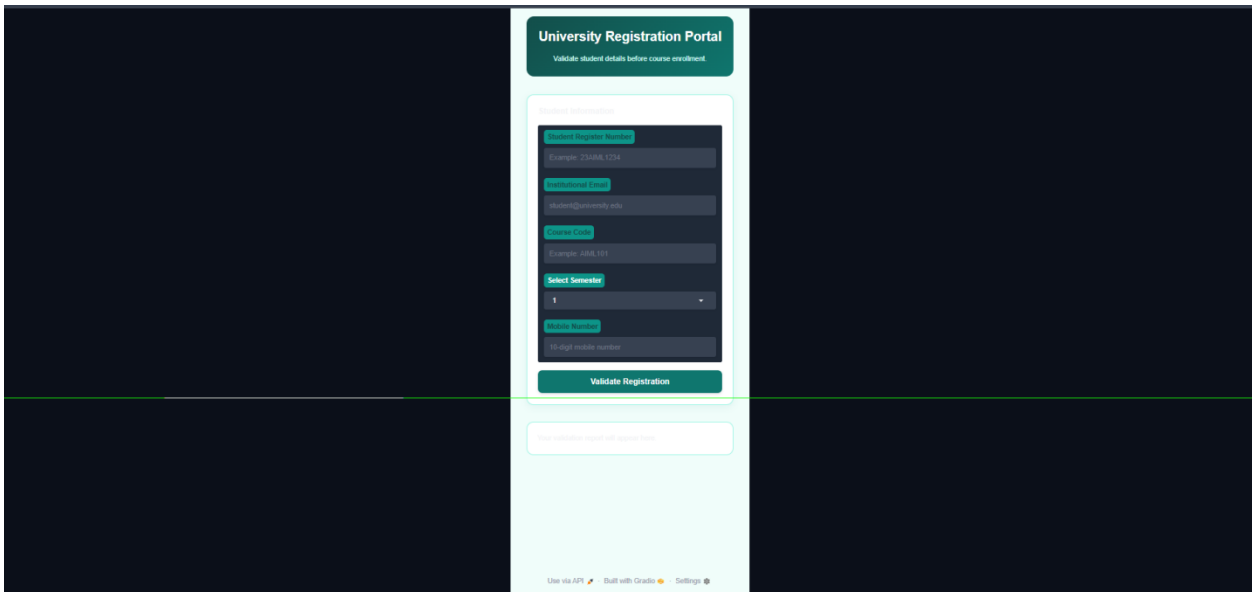
        submit_btn = gr.Button(
            "Validate Registration",
            elem_classes=["primary-btn"]
        )

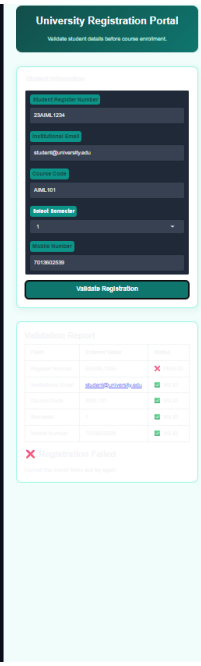
    with gr.Column(elem_id="result-box"):
        output = gr.Markdown(
            "Your validation report will appear here."
        )

    submit_btn.click(
        fn=validate_registration,
        inputs=[reg_no, email, course, semester, mobile],
        outputs=output
    )

demo.launch(share=True)
```

5. Sample Input




University Registration Portal
Validate student details before course enrollment.
Student Information
Student Register Number
23AIML1234
Institutional Email
student@university.edu
Course Code
AIML101
Select Semester
1
Mobile Number
7019602530
Validate Registration
Validation Report
Field
Entered Value
Status
Register Number
Invalid format
Invalid
Institutional Email
student@university.edu
Valid
Course Code
AIML101
Valid
Semester
1
Valid
Mobile Number
7019602530
Valid
Registration Failed
Correct the invalid fields and try again.